

Module 1: Python in Engineering & Fundamentals

- **Talk Track for Instructor:** Emphasize how L&T leverages Python for rapid prototyping, telemetry log parsing, and automating repetitive engineering reports. Review how core data types (lists, dictionaries, sets) map to real-world industrial entities like machine metadata, sensor registries, and error codes.
- **Core Code Concept – Efficient Data Handling:**

Mapping plant asset metadata and tracking states using advanced collections

```
from typing import Dict, Set

def filter_active_sensors(sensor_registry: Dict[str, dict], excluded_zones: Set[str]) -> Dict[str, dict]:
    active_sensors = {}
    for sensor_id, data in sensor_registry.items():
        if data["zone"] not in excluded_zones and data["status"] == "Online":
            active_sensors[sensor_id] = data
    return active_sensors

registry = {
    "S_01": {"zone": "Zone_A", "status": "Online", "temp": 74.2},
    "S_02": {"zone": "Zone_B", "status": "Maintenance", "temp": 82.1},
    "S_03": {"zone": "Zone_C", "status": "Online", "temp": 68.5}
}

print(filter_active_sensors(registry, {"Zone_B"}))
```

Module 2: Functions for Calculations & Tasks

- **Talk Track for Instructor:** Discuss how to write pure, testable functions for industrial calculations (e.g., stress analysis, pressure drop estimation). Explain variable scope and flexible parameter unpacking for extensible utility functions.
- **Core Code Concept – Flexible Parameters & Closures:**

```
def configure_telemetry_logger(prefix: str):
    def log_message(message: str) -> str:
        return f"[{prefix.upper()}] - {message}"
    return log_message

turbine_logger = configure_telemetry_logger("Turbine_Unit_01")
print(turbine_logger("RPM nominal reached."))
print(turbine_logger("Vibration warning flagged."))
```

Module 3: Clean Coding & Maintainable Scripts

- **Talk Track for Instructor:** Clean code saves hours in maintenance. Highlight the use of type hints (typing), docstrings (PEP 257), descriptive variable names, and explicit exception handling (try-except) instead of broad blanket catches.
- **Core Code Concept – Type Hints & Defensive Error Handling:**

```
from typing import List, Union

def calculate_average_pressure(readings: List[Union[int, float]]) -> float:
    """
    Calculate mean pressure from a list of sensor float/int readings.
    Raises ValueError if the input list is empty or contains invalid values.
    """
    if not readings:
        raise ValueError("Readings dataset cannot be empty.")
    total = 0.0
    for val in readings:
        if not isinstance(val, (int, float)):
            raise TypeError(f"Invalid reading type detected: {type(val)}")
        total += float(val)
    return total / len(readings)

try:
    print("Mean Pressure:", calculate_average_pressure([101.3, 102.5, 99.8]))
except (ValueError, TypeError) as e:
    print(f"Error handling sensor computation: {e}")
```

Part 2: Intermediate-to-Hard Coding Programs

Program 1: Industrial SCADA Log Stream Aggregator (Intermediate)

- **Problem Statement:** Write a Python function that processes a stream of raw plant logs formatted as [TIMESTAMP] [LEVEL] [MACHINE_ID] Message. Extract all ERROR logs, group them by MACHINE_ID, and return a dictionary containing the total error count and a chronologically sorted list of timestamp strings.
- **Code Solution:**

```
from collections import defaultdict
from datetime import datetime
```

```

from typing import Dict, List

def parse_industrial_logs(log_stream: List[str]) -> Dict[str, dict]:
    error_data = defaultdict(lambda: {"count": 0, "timestamps": []})
    for line in log_stream:
        parts = line.strip().split(" ", 3)
        if len(parts) < 4:
            continue
        timestamp_str, level, machine_id, _ = parts[0], parts[1], parts[2], parts[3]
        level = level.strip("[]")
        if level == "ERROR":
            dt = datetime.strptime(timestamp_str.strip("[]"), "%Y-%m-%d %H:%M:%S")
            error_data[machine_id]["count"] += 1
            error_data[machine_id]["timestamps"].append(dt)
    result = {}
    for machine, data in error_data.items():
        sorted_ts = sorted(data["timestamps"])
        result[machine] = {
            "count": data["count"],
            "timestamps": [ts.strftime("%Y-%m-%d %H:%M:%S") for ts in sorted_ts]
        }
    return result

logs = [
    "[2026-08-15 08:00:01] [INFO] [TURBINE_01] Startup sequence initiated",
    "[2026-08-15 08:15:22] [ERROR] [TURBINE_01] Bearing temperature high",
    "[2026-08-15 08:02:11] [ERROR] [PUMP_04] Pressure drop below threshold",
    "[2026-08-15 08:10:15] [ERROR] [TURBINE_01] Lubrication flow fault"
]

print(parse_industrial_logs(logs))

```

Program 2: Greedy Heavy Equipment Maintenance Scheduling (Intermediate-Hard)

- Problem Statement:** Given a list of maintenance time intervals represented as tuples (start_hour, end_hour), design a greedy algorithm to maximize the total number of non-overlapping maintenance tasks performed by a single engineering squad.

- **Code Solution:**

```
from typing import List, Tuple

def max_equipment_maintenance(tasks: List[Tuple[int, int]]) -> int:
    if not tasks:
        return 0

    sorted_tasks = sorted(tasks, key=lambda x: x[1])
    count = 1
    last_end_time = sorted_tasks[0][1]
    for start, end in sorted_tasks[1:]:
        if start >= last_end_time:
            count += 1
            last_end_time = end
    return count

schedule = [(8, 10), (9, 12), (11, 14), (13, 16), (15, 17)]
print(f'Maximum maintenance operations scheduled: {max_equipment_maintenance(schedule)}')
```

Program 3: Manufacturing Pipeline Assembly Dependency Resolution (Hard)

- **Problem Statement:** Given a dictionary representing structural sub-component dependencies in an EPC plant workflow, use Kahn's Algorithm (in-degree tracking) to return a valid assembly sequence. Raise a ValueError if a circular dependency loop occurs.
- **Code Solution:**

```
from collections import defaultdict, deque
from typing import Dict, List

def resolve_assembly_pipeline(dependencies: Dict[str, List[str]]) -> List[str]:
    in_degree = defaultdict(int)
    all_nodes = set(dependencies.keys())
    for node, deps in dependencies.items():
        for dep in deps:
            all_nodes.add(dep)
    for node in all_nodes:
        if node not in in_degree:
```

```

        in_degree[node] = 0
adj_list = defaultdict(list)
for node, deps in dependencies.items():
    for dep in deps:
        adj_list[dep].append(node)
        in_degree[node] += 1
queue = deque([node for node, degree in in_degree.items() if degree == 0])
result = []
while queue:
    current = queue.popleft()
    result.append(current)
    for neighbor in adj_list[current]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)
if len(result) != len(all_nodes):
    raise ValueError("Circular dependency detected in manufacturing workflow!")
return result
assembly_tree = {
    "Control_Panel": ["Wiring_Harness", "PLC_Unit"],
    "PLC_Unit": ["Power_Supply"],
    "Wiring_Harness": ["Power_Supply"],
    "Power_Supply": []
}
print("Optimal Assembly Sequence:", resolve_assembly_pipeline(assembly_tree))

```

Part 3: Assessment Section

Section A: 3 Assessment Coding Problems

1. IoT Sensor Sliding Window Peak Detector:

- *Problem:* Write a function `find_peak_readings(readings: List[int], k: int) -> List[int]` that processes stream readings from heavy infrastructure monitors and finds the maximum value for every contiguous sliding window of size `k` in $O(n)$ time complexity using a deque.

2. Recursive Plant Configuration Merger:

- *Problem:* Write a function `merge_site_configs(base: dict, override: dict) -> dict` that recursively merges plant parameter dictionaries. Nested dictionaries must merge deeply, while standard key-values from override take precedence.

3. Assembly Line Maximum Profit Sub-segment:

- *Problem:* Given an array of integers tracking financial net outputs or efficiency gains across sequential stations in an L&T construction layout, write a function `max_segment_gain(gains: List[int]) -> int` to locate the continuous sub-segment yielding the maximum cumulative sum (Kadane's algorithm).

Section B: 10 Output-Based Questions

1. **Question:** What is the output of the following configuration accumulator function?

```
def append_sensor(sensor_id, registry=[]):  
    registry.append(sensor_id)  
    return registry  
print(append_sensor("S1"))  
print(append_sensor("S2"))
```

Answer: ['S1'] followed by ['S1', 'S2']. (*Default mutable arguments are evaluated once at definition time*).

2. **Question:** What will be printed by this snippet?

```
voltage = 230  
def check_status():  
    print(voltage)  
    voltage = 415  
check_status()
```

Answer: UnboundLocalError: local variable 'voltage' referenced before assignment. (*Assigning inside makes Python treat it as local*).

3. **Question:** What is the output of slice evaluation on component lists?

```
components = ["Valve", "Pump", "Turbine", "Boiler"]  
print(components[1:10:2])
```

- **Answer:** ['Pump', 'Boiler']. (*Slice indexes safe-clip to list bounds*).

4. **Question:** Evaluate the execution outcome:

```
line_a = [100, 200, 300]
line_b = line_a
line_b.append(400)
print(line_a is line_b, len(line_a))
```

- **Answer:** True 4. *(Assignment copies references, not underlying objects).*

5. **Question:** What is the output of this generator-based telemetry stream?

```
gen = (x * 10 for x in range(3))
print(sum(gen))
print(list(gen))
```

- **Answer:** 20 followed by []. *(Generators are single-pass iterators).*

6. **Question:** What does this print?

```
print(bool([]), bool("True"), bool(0.0))
```

- **Answer:** False True False. *(Non-empty strings are truthy; empty collections/zeros are falsy).*

7. **Question:** Determine the argument unpacking counts:

```
def inspect_telemetry(device_id, *args, **kwargs):
    print(len(args), len(kwargs))
inspect_telemetry("DEV_01", 10, 20, status="active", mode="auto", priority=1)
```

- **Answer:** 2 3. *(Positional args count = 2, keyword args count = 3).*

8. **Question:** What is the output data type evaluation?

```
print(type(400 / 2), type(400 // 2))
```

- **Answer:** <class 'float'> <class 'int'>. *(Standard division returns float; floor division preserves int).*

9. **Question:** What does this string slice return?

```
tag = "LT-PROJECT"
print(tag[::-1])
```

- **Answer:** TCETORP-TL. (*Negative step size reverses string characters*).

10. **Question:** Evaluate equality vs identity output:

```
print(50 == 50.0, 50 is 50.0)
```

- **Answer:** True False. (*== checks value equality; is tests object memory identity*).

Section C: 10 Multiple Choice Questions (MCQs)

1. **Which PEP standard governs official Python code style guidelines suited for enterprise scripts?**
 - A) PEP 20
 - B) PEP 8
 - C) PEP 257
 - D) PEP 3107
 - **Answer: B) PEP 8**
2. **What is the average time complexity for key lookups in a Python dictionary (dict)?**
 - A) $O(n)$
 - B) $O(\log n)$
 - C) $O(1)$
 - D) $O(n \log n)$
 - **Answer: C) $O(1)$**
3. **How do you denote a type hint that accepts either an integer or None in Python?**
 - A) val: int?
 - B) val: Optional[int] or val: int | None
 - C) val: Union(int, None)
 - D) val: int = None
 - **Answer: B) Optional[int] or val: int | None**
4. **Which module is part of the standard library for data structuring and array manipulations?**
 - A) math
 - B) array
 - C) itertools

- D) os
 - **Answer: B) array**
5. What is the primary role of an `__init__.py` file inside an engineering project module directory?
- A) Boost script compilation speed
 - B) Designate the directory as a recognized Python package
 - C) Store runtime error stacks
 - D) Configure global environment variables
 - **Answer: B) Designate the directory as a recognized Python package**
6. Which statement concerning Python tuples is factually correct?
- A) Tuples are mutable.
 - B) Tuples are immutable and can be used as dictionary keys if elements are hashable.
 - C) Tuples consume more memory than lists of identical length.
 - D) Tuples do not permit slicing operations.
 - **Answer: B) Tuples are immutable and can be used as dictionary keys if elements are hashable.**
7. What is the evaluated type returned by `type(lambda x: x * 2)`?
- A) `<class 'function'>`
 - B) `<class 'lambda'>`
 - C) `<class 'method'>`
 - D) `<class 'object'>`
 - **Answer: A) `<class 'function'>`**
8. Which keyword handles runtime exceptions robustly in Python scripts?
- A) catch
 - B) except
 - C) handle
 - D) trap
 - **Answer: B) except**
9. What data structure type does the built-in `zip()` function yield in Python 3?
- A) A list of tuples
 - B) A dictionary collection
 - C) An iterator of tuples
 - D) A flattened set

- **Answer: C) An iterator of tuples**

10. Which built-in function lists all attributes and methods available on an engineering object instance?

- A) inspect()
- B) attributes()
- C) dir()
- D) vars()
- **Answer: C) dir()**